AGENT TECHNOLOGY SHARING

Agent 做决策，一定要生成文字吗？

从 Jev 到 Perplexity Decider：用实际例子理解 System-One Models

JEV　SYSTEM-ONE　PPLX DECIDER

超博

2026.10 · 组会技术分享

先看一个大家熟悉的 Agent 任务

我们想找一篇论文的官方代码。这里先只研究“下一步用哪个工具”。

用户：帮我找一下 ReAct 论文的官方代码仓库。

Agent 当前有 3 个选择

A 搜索 GitHub

github_search

B 搜索论文网站

paper_search

C 直接回答用户

answer

接下来只有一个小问题：选 A、B，还是 C?

传统做法：让 LLM 写出一条工具调用

假设模型决定搜索 GitHub，它通常需要把调用内容生成出来。

LLM 输出的工具调用（示意）

```
{
    "tool": "github_search",
    "arguments": {
     "query": "ReAct paper official code"
    }
}
```

它要逐步写出...

1. 用哪个工具
2. 参数字段名
3. 搜索关键词
4. 完整调用结构

这里按“片段”展示；真实 LLM 是逐 token 生成，并非逐行一次写完。

但如果“候选动作和参数”已经准备好了呢?

此时我们只需要选中一个动作，剩下的可以由程序执行。

已准备好的内容

由程序 / 外层 Agent 提供

候选动作：A / B / C

搜索词：ReAct paper official code

工具参数模板：已定义

真正需要 AI 判断的是：

“现在应该选哪一个？”

Jev 适合这种“固定选项 + 上下文判断”的场景；它不会凭空写出新的搜索词。

Jev：不写调用文本，直接给出“怎么选”

输入当前任务和候选选项，输出每个选项的概率。

输入

任务：找 ReAct 官方代码

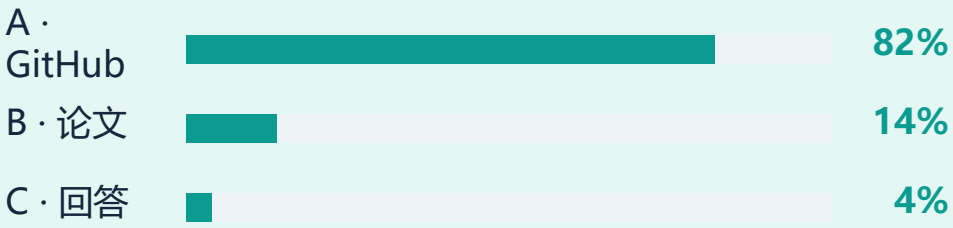
A GitHub 搜索

B 论文网站

C 直接回答

输出：Choice

选 A：GitHub 搜索



程序拿到 A 之后，按预先写好的模板调用 github_search。

资料：TypeSafe 官方介绍

同一个选择，LLM 和 Jev 的差别在哪?

重点不是“A 这个字符”；重点是模型被设计成“写内容”还是“选答案”。

生成式 LLM

我要把工具调用写出来

```
{"tool":"github_search", ...}
```

可以自由生成新的文本 / 参数

Jev / System-One

我要从预定义答案中选择

A 82%　B 14%　C 4%

无需生成任意工具调用文本

LLM 也可以用受约束输出或候选 logits；这里比较的是典型接口和专门优化目标。

做完 GitHub 搜索后，Agent 还要判断什么?

同一份搜索结果，可以对应三种很常见的决策问题。

搜索结果：找到 3 个仓库，第一个 README 提到了 ReAct 论文。

Choice • 选一个

下一步做什么?

打开仓库 • 74%

其他：继续搜索 / 回答

Score • 打个分

结果质量如何?

较好 • 高概率

评分：低 / 中 / 高

Noul • 是或否

还需要人工核验?

是 • 81%

直接输出 P(是)

示例答案只用于帮助理解；Noul 可以简单理解成 Yes/No 概率。

资料：TypeSafe API

“Jev 可以并行”到底是什么意思?

想象把同一份搜索结果交给一个助手，一口气问三道题。

共同的 State

GitHub 返回了 3 个仓库

其中 1 个可能是

ReAct 官方实现

（只提供一次）

问题 1：下一步做什么? → **Choice：打开仓库**

问题 2：质量如何? → **Score：较好**

问题 3：要不要验证? → **Noul：81% 是**

一次 API 请求带上多个问题 → 一次响应收到多个结构化答案。

资料：TypeSafe API 一次接受多个 questions

“一次问多题”不等于“所有思考都自动完成”

三个问题可以一起提交，但它们仍需要外层 Agent 组织和检查。

Jev 擅长

已知要问什么

已知允许选哪些答案

一次返回多个判断结果

仍需要代码 / LLM 负责

决定应该问哪些问题

生成新的计划或搜索词

处理相互矛盾的判断

“并行”不代表内部必定只做一次前向计算，也不代表三个答案天然逻辑一致。

为什么它可能更快、更省?

如果每天要做几十万次小判断，少生成一些无关内容就可能很有价值。

传统 LLM

判断 → 写出文本 → 解析

System-One

判断 → 返回结构化结果

真实案例：招聘匹配环节使用 Jev，厂商报告成本降低 88%、处理时间约减半。

结果是特定业务的对比；不能据此推断所有 Agent 都会加速 2 倍。

资料：TypeSafe 招聘匹配应用案例

下一个问题

Jev 的模型内部，到底是怎么做的?

Jev：完整网络结构仍未公开

Perplexity：有可分析的开源决策模型

用 Perplexity Decider 解释：怎样把现有 LLM 改造成“专门做选择”的模型。

Perplexity 做了一件很直接的事：改造 Qwen

保留擅长理解上下文的模型主体，改造“最后怎样给答案”。

原来的 Qwen

理解任务 / 上下文

写出任意文字、代码、JSON…

PPLX Decider

Qwen 主体：理解任务 / 上下文

从固定候选项中输出概率

理解能力从预训练模型继承，输出方式和训练目标再做专门化。

资料：PPLX Decider v1.1 模型卡

它怎么从“看到问题”变成“选择 A”？

继续用刚才的工具选择题，看一遍实际的决策流程。

① 给出问题

ReAct 代码在哪里找?
A / B / C

② Qwen 理解

读懂任务、上下文
以及各个选项

③ 决策读出层

不预测所有词
只计算候选分数

④ 输出概率

A 82% B 14%
C 4%

模型细节：最后位置的隐藏表示（5120 维）→ 决策读出（最多 255 个位置）→ 候选项概率。

255 是读出位置上限，不是说每道题都有 255 个选项；这里仅有 A / B / C。

资料：PPLX v1.1 模型卡、NOTICE

为什么 PPLX 还改了 Attention?

生成文字要按顺序往后写；做判断时，更希望每个位置都能利用整段输入。

生成式阅读（Causal）

看“论文”这个位置时

帮 我 找 论 文 代 码

只能读当前位置及其之前的信息

非因果阅读（Noncausal）

看“论文”这个位置时

帮 我 找 论 文 代 码

能同时利用后面的“代码”信息

PPLX v1.1 只改了部分 Full-Attention 层；并非整个模型都改成双向。

资料：PPLX v1.1 Attention 说明

有了 Decision Head，还得教模型“怎么选”

可以把它想象成：给模型大量带标准答案的选择题。

工具选择	任务：找论文的代码	答案：A • GitHub
状态判断	任务：结果是否足够	答案：是 / 否
分级评分	任务：搜索质量如何	答案：低 / 中 / 高

训练时在做什么?

预测一个答案

↓

与标准答案比较

↓ 更新权重

PPLX 明确披露了监督微调（SFT）；不能直接假设其使用了 Jev 的 RLCD。

资料：Perplexity 模型卡、NOTICE

模型说“80% 确定”，到底意味着什么?

好的概率输出，不只是哪个选项排第一，还应该大致知道自己有多可能答对。

模型做了 10 道题

每题都自称

“正确概率约 80%”

那么统计上希望.....

这 10 次中，大约 8 次真的答对



绿色：正确　　灰色：错误

校准看的是“许多类似预测的统计表现”，不是保证单个 80% 的答案一定正确。

Perplexity 改完以后，实际成绩怎样？

Decision Index 是一个专门的决策评测套件：分数越高越好。



在这套评测上：PPLX v1.1 高于 Jev，但不代表所有任务、所有 Agent 都更好。

资料：Perplexity v1.1 模型卡

真正放进 Agent，可以怎样配合？

LLM 负责开放式的规划与生成；Decider 负责“已定义选项中的高频判断”。

① 用户任务
找 ReAct 代码

② LLM / 代码
准备搜索词

③ 搜索工具
返回仓库列表

④ Decider
下一步怎么选?

⑤ Agent 执行
打开 / 核验 / 回答

Decider 输出示例

Choice
打开仓库

Noul
需要核验

Choice
核验后回答

它适合做什么，不适合做什么？

判断一个场景能否交给 Jev，最简单的办法是看“答案是不是提前定义好的”。

适合：固定候选答案

- ✓ 选 GitHub 还是论文网站？
- ✓ 结果够好了吗？
- ✓ 是否需要重试？
- ✓ 这个回答质量是高 / 中 / 低？

不适合直接代替：开放式生成

- × 重新写一段程序
- × 生成一条全新的 Shell 命令
- × 设计完整的多步研究方案
- × 创造新的工具参数或选项

如果正确答案不在候选集中，模型再“自信”也可能只能选错。

更进一步：Agent 一定要用一个 LLM 包办全部思考吗?

Jev 提供了一个启发：不同问题，可能对应不同的专门“认知工具”。

常见做法

一个 LLM

规划 / 决策 / 验证 / 生成

一种值得探索的做法

写内容	Generator
做选择	Decider
检查对错	Verifier
决定下一步	Controller

真正值得研究：什么时候调用哪个模型？多加一个模块到底能否提升长期任务效果?

总结

01

LLM 擅长“写出来”

Jev 擅长“从已给选项中选出来”

02

Jev 可以一次回答多个已定义问题

但不能代替开放式规划与工具参数生成

03

Perplexity 给出了可研究的实现

Qwen 主体 + 决策读出层 + 专门训练与校准

THANK YOU • Q&A

参考资料

技术细节以官方发布与模型卡为准；示例流程和概率均为讲解示意。

TypeSafe — Jev 官方介绍	System-One 的定位、非自回归与 RLCD
TypeSafe — API Reference	同一请求中的 state + 多个 questions
Perplexity — Decider v1.1	架构、评测、255 维 readout 和非因果注意力
Perplexity — NOTICE	SFT、last-token pooling、单独拟合校准温度
TypeSafe — Jack & Jill 案例	生产案例的数据与对比条件

特别说明：Jev 的完整网络结构没有公开；Perplexity 是同类模型的公开实现，不能等同于 Jev 本身。